

Keywords and operators

June 17, 2026 3:19 PM

<https://kotlinlang.org/docs/keyword-reference.html>

Keywords and operators

Hard keywords

Soft keywords

Modifier keywords

Special identifiers

Operators and special symbols

Keywords and operators

 [Edit page](#) Last modified: 28 May 2026

Hard keywords

The following tokens are always interpreted as keywords and cannot be used as identifiers:

- `as`
 - is used for type casts.
 - specifies an alias for an import
- `as?` is used for safe type casts.
- `break` terminates the execution of a loop.
- `class` declares a class.
- `continue` proceeds to the next step of the nearest enclosing loop.
- `do` begins a do/while loop (a loop with a postcondition).
- `else` defines the branch of an if expression that is executed when the condition is false.
- `false` specifies the 'false' value of the Boolean type.
- `for` begins a for loop.
- `fun` declares a function.
- `if` begins an if expression.
- `in`
 - specifies the object being iterated in a for loop.
 - is used as an infix operator to check that a value belongs to a range, a collection, or another entity that defines a 'contains' method.
 - is used in when expressions for the same purpose.
 - marks a type parameter as contravariant.
- `!in`
 - is used as an operator to check that a value does NOT belong to a range, a collection, or another entity that defines a 'contains' method.
 - is used in when expressions for the same purpose.
- `interface` declares an interface.
- `is`
 - checks that a value has a certain type.
 - is used in when expressions for the same purpose.

- `!is`
 - checks that a value does NOT have a certain type.
 - is used in when expressions for the same purpose.
- `null` is a constant representing an object reference that doesn't point to any object.
- `object` declares a class and its instance at the same time.
- `package` specifies the package for the current file.
- `return` returns from the nearest enclosing function or anonymous function.
- `super`
 - refers to the superclass implementation of a method or property.
 - calls the superclass constructor from a secondary constructor.
- `this`
 - refers to the current receiver.
 - calls another constructor of the same class from a secondary constructor.
- `throw` throws an exception.
- `true` specifies the 'true' value of the Boolean type.
- `try` begins an exception-handling block.
- `typealias` declares a type alias.
- `typeof` is reserved for future use.
- `val` declares a read-only property or local variable.
- `var` declares a mutable property or local variable.
- `when` begins a when expression (executes one of the given branches).
- `while` begins a while loop (a loop with a precondition).

Soft keywords

The following tokens act as keywords in the context in which they are applicable, and they can be used as identifiers in other contexts:

- `by`
 - delegates the implementation of an interface to another object.
 - delegates the implementation of the accessors for a property to another object.
- `catch` begins a block that handles a specific exception type.
- `constructor` declares a secondary constructor.

Soft keywords

The following tokens act as keywords in the context in which they are applicable, and they can be used as identifiers in other contexts:

- `by`
 - delegates the implementation of an interface to another object.
 - delegates the implementation of the accessors for a property to another object.
- `catch` begins a block that handles a specific exception type.
- `constructor` declares a primary or secondary constructor.
- `delegate` is used as an annotation use-site target.
- `dynamic` references a dynamic type in Kotlin/JS code.
- `field`
 - declares an explicit backing field.
 - is used as an annotation use-site target.
- `file` is used as an annotation use-site target.
- `finally` begins a block that is always executed when a try block exits.
- `get`
 - declares the getter of a property.
 - is used as an annotation use-site target.
- `import` imports a declaration from another package into the current file.
- `init` begins an initializer block.
- `param` is used as an annotation use-site target.
- `property` is used as an annotation use-site target.
- `receiver` is used as an annotation use-site target.
- `set`
 - declares the setter of a property.
 - is used as an annotation use-site target.
- `setparam` is used as an annotation use-site target.
- `value` with the `class` keyword declares an inline class.
- `where` specifies the constraints for a generic type parameter.

Modifier keywords

The following tokens act as keywords in modifier lists of declarations, and they can be used

Modifier keywords

The following tokens act as keywords in modifier lists of declarations, and they can be used as identifiers in other contexts:

- `abstract` marks a class or member as abstract.
- `actual` denotes a platform-specific implementation in multiplatform projects ↗.
- `annotation` declares an annotation class.
- `companion` declares a companion object.
- `const` marks a property as a compile-time constant.
- `crossinline` forbids non-local returns in a lambda passed to an inline function.
- `data` instructs the compiler to generate canonical members for a class.
- `enum` declares an enumeration.
- `expect` marks a declaration as platform-specific ↗, expecting an implementation in platform modules.
- `external` marks a declaration as implemented outside of Kotlin (accessible through JNI or in JavaScript).
- `final` forbids overriding a member.
- `infix` allows calling a function using infix notation.
- `inline` tells the compiler to inline a function and the lambdas passed to it at the call site.
- `inner` allows referring to an outer class instance from a nested class.
- `internal` marks a declaration as visible in the current module.
- `lateinit` allows initializing a non-nullable property outside of a constructor.
- `noinline` turns off inlining of a lambda passed to an inline function.
- `open` allows subclassing a class or overriding a member.
- `operator` marks a function as overloading an operator or implementing a convention.
- `out` marks a type parameter as covariant.
- `override` marks a member as an override of a superclass member.
- `private` marks a declaration as visible in the current class or file.
- `protected` marks a declaration as visible in the current class and its subclasses.
- `public` marks a declaration as visible anywhere.
- `reified` marks a type parameter of an inline function as accessible at runtime.
- `sealed` declares a sealed class (a class with restricted subclassing).
- `suspend` marks a function or lambda as suspending (usable as a coroutine).
- `tailrec` marks a function as tail-recursive (allowing the compiler to replace recursion with iteration).

- `tailrec` marks a function as tail-recursive (allowing the compiler to replace recursion with iteration).
- `vararg` allows passing a variable number of arguments for a parameter.

Special identifiers

The following identifiers are defined by the compiler in specific contexts, and they can be used as regular identifiers in other contexts:

- `field` is used inside a property accessor to refer to the backing field of the property.
- `it` is used inside a lambda to refer to its parameter implicitly.

Operators and special symbols

Kotlin supports the following operators and special symbols:

- `+`, `-`, `*`, `/`, `%` - mathematical operators
 - `*` is also used to pass an array to a `vararg` parameter.
- `=`
 - assignment operator.
 - is used to specify default values for parameters.
- `+=`, `--`, `*=`, `/=`, `%=` - augmented assignment operators.
- `++`, `--` - increment and decrement operators.
- `&&`, `||`, `!` - logical 'and', 'or', 'not' operators (for bitwise operations, use the corresponding infix functions instead).
- `==`, `!=` - equality operators (translated to calls of `equals()` for non-primitive types).
- `===`, `!==` - referential equality operators.
- `<`, `>`, `<=`, `>=` - comparison operators (translated to calls of `compareTo()` for non-primitive types).
- `[ , ]` - indexed access operator (translated to calls of `get` and `set`).
- `!!` asserts that an expression is non-nullable.
- `?.` performs a safe call (calls a method or accesses a property if the receiver is non-nullable).
- `?:` takes the right-hand value if the left-hand value is null (the elvis operator).
- `::` creates a member reference or a class reference.
- `..`, `..<` create ranges.
- `:` separates a name from a type in a declaration.

- `:` separates a name from a type in a declaration.
- `?` marks a type as nullable.
- `->`
 - separates the parameters and body of a lambda expression.
 - separates the parameters and return type declaration in a function type.
 - separates the condition and body of a when expression branch.
- `@`
 - introduces an annotation.
 - introduces or references a loop label.
 - introduces or references a lambda label.
 - references a 'this' expression from an outer scope.
 - references an outer superclass.
- `;` separates multiple statements on the same line.
- `$` references a variable or expression in a string template.
- `_`
 - substitutes an unused parameter in a lambda expression.
 - substitutes an unused parameter in a destructuring declaration.

For operator precedence, see [this reference ↗](#) in Kotlin grammar.